

Clase 1: Sistemas de Numeración

Binario : Van desde 0 a $2^n - 1$, 0 = 0V, 1 = 5V

$n = 4 \rightarrow 4 \text{ bits} = 1 \text{ nibble}$

$n = 8 \rightarrow 8 \text{ bits} = 1 \text{ byte}$

$n = 16 \rightarrow 16 \text{ bits} = 1 \text{ word}$

Suma ponderada : Multiplicamos cada bit por el peso de su posición (2^i) y luego sumamos todo.
(binario a decimal)

$$11011_B = \underbrace{1 \cdot 2^4}_{16} + \underbrace{1 \cdot 2^3}_{8} + \underbrace{0 \cdot 2^2}_0 + \underbrace{1 \cdot 2^1}_4 + \underbrace{1 \cdot 2^0}_1 = 27_D$$

$$11111111_B = \underbrace{1 \cdot 2^7}_{128} + \underbrace{1 \cdot 2^6}_{64} + \underbrace{1 \cdot 2^5}_{32} + \underbrace{1 \cdot 2^4}_{16} + \underbrace{1 \cdot 2^3}_{8} + \underbrace{1 \cdot 2^2}_{4} + \underbrace{1 \cdot 2^1}_{2} + \underbrace{1 \cdot 2^0}_1 = 255_D = 2^8 - 1$$

mini ejercicios de la filmina

$$011111110011_B = \underbrace{0 \cdot 2^{11}}_{2048} + \underbrace{1 \cdot 2^{10}}_{1024} + \underbrace{1 \cdot 2^9}_{512} + \underbrace{1 \cdot 2^8}_{256} + \underbrace{1 \cdot 2^7}_{128} + \underbrace{1 \cdot 2^6}_{64} + \underbrace{1 \cdot 2^5}_{32} + \underbrace{1 \cdot 2^4}_{16} + \underbrace{0 \cdot 2^3}_0 + \underbrace{0 \cdot 2^2}_0 + \underbrace{1 \cdot 2^1}_2 + \underbrace{1 \cdot 2^0}_1 = 4095_D$$

$$0001010101100000_B = \underbrace{1 \cdot 2^{12}}_{4096} + \underbrace{1 \cdot 2^{10}}_{1024} + \underbrace{1 \cdot 2^8}_{512} + \underbrace{1 \cdot 2^6}_{64} + \underbrace{1 \cdot 2^5}_{32} = 5856_D$$

$$\underbrace{111111111111}_\text{todos 1}_B = 2^{12} - 1 = 4095_D$$

División Repetida : Consiste en dividir un numero decimal entero sucesivamente por 2 e ir registrando los restos.
(decimal a binario)

Siendo Q el cociente : Si $Q \neq 0 \rightarrow$ Sigo dividiendo entre 2

Si $Q = 0 \rightarrow$ Reuno los restos, el primer resto es LSB y el ultimo cociente MSB

$$\begin{array}{r} 128 \overline{) 128} \\ 0 \\ \hline 64 \overline{) 64} \\ 0 \\ \hline 32 \overline{) 32} \\ 0 \\ \hline 16 \overline{) 16} \\ 0 \\ \hline 8 \overline{) 8} \\ 0 \\ \hline 4 \overline{) 4} \\ 0 \\ \hline 2 \overline{) 2} \\ 0 \\ \hline 1 \end{array}$$

$10000000_B = 128_D$

LSB $\rightarrow$ MSB

$$\begin{array}{r} 555 \overline{) 555} \\ 554 \\ \hline 1 \\ 277 \overline{) 277} \\ 276 \\ \hline 1 \\ 138 \overline{) 138} \\ 136 \\ \hline 2 \\ 69 \overline{) 69} \\ 68 \\ \hline 1 \\ 34 \overline{) 34} \\ 32 \\ \hline 2 \\ 17 \overline{) 17} \\ 16 \\ \hline 1 \\ 8 \overline{) 8} \\ 8 \\ \hline 0 \\ 4 \overline{) 4} \\ 4 \\ \hline 0 \\ 2 \overline{) 2} \\ 2 \\ \hline 0 \\ 1 \end{array}$$

$1101010001_B = 555_D$

mini ejercicios de la filmina

$$\begin{array}{r}
 2040 \div 2 = 1020 \\
 1020 \div 2 = 510 \\
 510 \div 2 = 255 \\
 255 \div 2 = 127 \\
 127 \div 2 = 63 \\
 63 \div 2 = 31 \\
 31 \div 2 = 15 \\
 15 \div 2 = 7 \\
 7 \div 2 = 3 \\
 3 \div 2 = 1 \\
 1 \div 2 = 0
 \end{array}$$

1111111000₂ = 2040₁₀

$$\begin{array}{r}
 4095 \div 2 = 2047 \\
 2047 \div 2 = 1023 \\
 1023 \div 2 = 511 \\
 511 \div 2 = 255 \\
 255 \div 2 = 127 \\
 127 \div 2 = 63 \\
 63 \div 2 = 31 \\
 31 \div 2 = 15 \\
 15 \div 2 = 7 \\
 7 \div 2 = 3 \\
 3 \div 2 = 1 \\
 1 \div 2 = 0
 \end{array}$$

11111111111₂ = 4095₁₀

Hexadecimal : Sirve para "abreviar" cadenas de bits . Van desde 0 a $16^n - 1$. La posición se incrementa en 1 cada dígito , desde 0 hasta F

H	D	Binario de 4 bits			
0	0	0	0	0	0
1	1	0	0	0	1
2	2	0	0	1	0
3	3	0	0	1	1
4	4	0	1	0	0
5	5	0	1	0	1
6	6	0	1	1	0
7	7	0	1	1	1
8	8	1	0	0	0
9	9	1	0	0	1
A	10	1	0	1	0
B	11	1	0	1	1
C	12	1	1	0	0
D	13	1	1	0	1
E	14	1	1	1	0
F	15	1	1	1	1

Decodificar bits : Cada dígito en Hexa equivale a un bloque exacto de 4 bits

$$A5_H : 1010\ 0101_B$$

Suma ponderada : Multiplicamos cada bit por el peso de su posición (16^i) y luego sumamos todo.

$$7F3_H = \underbrace{7 \cdot 16^2}_{1792} + \underbrace{F \cdot 16^1}_{240} + \underbrace{3 \cdot 16^0}_3 = 2035_D$$

$$FFF_H = \underbrace{F \cdot 16^2}_{3840} + \underbrace{F \cdot 16^1}_{240} + \underbrace{F \cdot 16^0}_{15} = 4095_D$$

mini ejercicios de la filmina

$$1560_H = \underbrace{1 \cdot 16^3}_{4096} + \underbrace{5 \cdot 16^2}_{1280} + \underbrace{6 \cdot 16^1}_{96} + \underbrace{0 \cdot 16^0}_0 = 5472_D$$

$$9A7_H = \underbrace{9 \cdot 16^2}_{2304} + \underbrace{A \cdot 16^1}_{160} + \underbrace{7 \cdot 16^0}_7 = 2471_D$$

División Repetida (decimal a Hexa) : Consiste en dividir un numero decimal entero sucesivamente por 16 e ir registrando los restos.

Si $Q > 16 \rightarrow$ Sigo dividiendo entre 16
 Si $Q < 16 \rightarrow$ Reuno los restos, el primer resto es LSD y el ultimo cociente MSD

$$\begin{array}{r} 128 \overline{) 16} \\ \underline{0} = 8 \end{array}$$

MSD $\rightarrow$ LSD

80_H

mini ejercicios de la filmina

$$\begin{array}{r} 555 \overline{) 16} \\ \underline{544} = 34 \overline{) 16} \\ \underline{32} = 2 \\ \underline{2} \end{array}$$

228_H

$$\begin{array}{r} 2040 \overline{) 16} \\ \underline{2032} = 127 \overline{) 16} \\ \underline{8} = 7 \\ \underline{15} \end{array}$$

7F8_H

Hexadecimal a Binario : Cada digito hexadecimal tiene un peso equivalente a una potencia de 16

7F3_H = $\underbrace{0111}_7 \underbrace{1111}_F \underbrace{0011}_3$ B

FFF_H = 1111 1111 1111 B

mini ejercicios de la filmina

1560_H = $\underbrace{0001}_1 \underbrace{0101}_5 \underbrace{0110}_6 \underbrace{0000}_0$ B

9AG_H = 1001 1010 0110 B

operaciones con sistemas Hexadecimales

Para la resta : cuando el digito de arriba es más chico que el de abajo, le saco 16 al de al lado

Para la suma : cuando la suma de ambos digitos da mayor a 16, le restamos 16 y me llevo 1

$$\begin{array}{r} FFFF_H \\ - 3D00 \\ \hline C2FF \end{array}$$

mini ejercicios de la filmina

$$\begin{array}{r} +16 +16 +16 \\ C A 5 5_H \\ - B 9 4 \\ \hline B B 7 7_H \\ = 0 E 0 E_H \end{array}$$

$$\begin{array}{r} -16 \\ 1D99_H \\ + F1C3_H \\ \hline 10F5C \end{array}$$

cout

$$\begin{array}{r} +16 \\ 8A7F_H \\ - 79E6_H \\ \hline 1099_H \end{array}$$

Código BCD

Para asignar un signo usamos una codificación de 4 bits :

POSITIVO (+) : 1100_B $\approx$ C_H NEGATIVO (-) : 1101_B $\approx$ B_H

BCD Desempaquetado

Se utiliza un byte entero para representar un solo digito decimal .

- Los 4 bits menos significativos (la mitad derecha) **contemplan el número**.
- Los 4 bits más significativos (la mitad izquierda) **contemplan el signo**.
- Ponemos el signo (+ó-) solo en el **último digito** del número completo. Para el resto de digitos se rellena con 1111 .

$$-834_H \rightarrow \begin{array}{|c|c|c|c|c|c|} \hline 1111 & 1000 & 1111 & 0011 & 1101 & 0100 \\ \hline F & 8 & F & 3 & (-) & 4 \\ \hline \end{array}$$

1 byte 1 byte 1 byte

BCD Empaquetado

Se utiliza un nibble entero para representar un solo dígito decimal.

- Los 4 bits menos significativos (la mitad derecha) **contemplan el primer dígito**.
- Los 4 bits más significativos (la mitad izquierda) **contemplan el segundo dígito**.
- Ponemos el signo (+ó-) en un nibble extra al final.

$$-834_H = \begin{array}{|c|c|c|c|} \hline 1000 & 0011 & 0100 & 1101 \\ \hline 8 & 3 & 4 & (-) \\ \hline \end{array}_B$$

1 byte 1 byte

mini ejercicios de la filmina

$D \rightsquigarrow BCD_E$

$$98_D = 1001 \ 1000 \ 1100_B$$

$$63_D = 0110 \ 0011 \ 1100_B$$

$$33_D = 0011 \ 0011 \ 1100_B$$

$D \rightsquigarrow BCD_D$

$$98_D = 1111 \ 1001 \ 1100 \ 1000_B$$

$$63_D = 1111 \ 0110 \ 1100 \ 0011_B$$

$$33_D = 1111 \ 0011 \ 1100 \ 0011_B$$

Mapeo de Memorias

Es el proceso de dividir el espacio de direcciones del procesador físico reales de memoria. Los buses del CPU son bus de direcciones, bus de datos y bus de control.

Tipo de Mapeos:

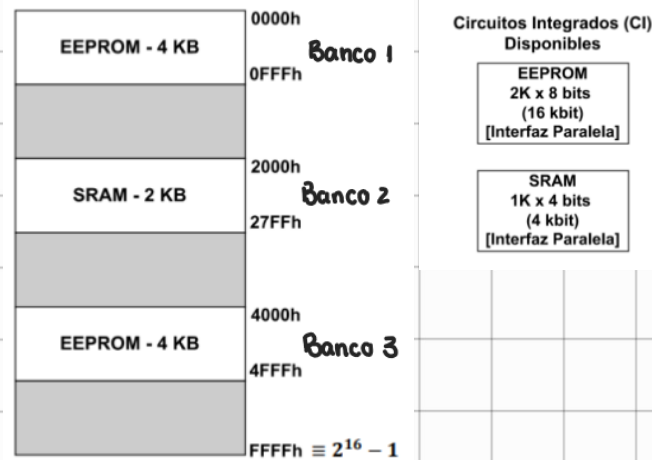
* Mapeo Absoluto: Todas las líneas de direcciones no asignadas a las memorias se utilizan en la decodificación externa. No existen zonas "reflejadas" ni sombras.

* Mapa Relativo: Quedan líneas del bus de direcciones sueltas o sin conectar en los decodificadores.

Esto genera que los bancos de memoria aparezcan **reflejados (repetidos)** en otras zonas no asignadas del mapa de memoria (2^n reflexiones, donde n es la cantidad de líneas libres).

Ejercicio de Mapeo Directo

Se dispone de un procesador de 16 bits de bus de direcciones y se debe implementar un Banco de Memoria que responda al siguiente mapa



PASO ① Almacena Disponible de los CI

tengo EEPROM 2K x 8 bits y SRAM 1K x 4 bits

Calculo de EEPROM

$$E: 2K \times 8 \rightarrow 2 \cdot (1KB) = 2 \cdot (1024_D) = 2048_D$$

$$\rightarrow 2048_D = 800_H$$

$$N_E^A = 800_H \text{ (ubicaciones de memoria)}$$

$$N_E^D = 8 \text{ bits (tamaño de cada ubicación de memoria)}$$

CA

$$\begin{array}{r} 2048 \overline{) 16} \\ 0 : 128 \overline{) 16} \\ 0 : 8 \end{array}$$

Calculo de SRAM

$$SR: 1K \times 4 \rightarrow N_{SR}^A = 1 \cdot (1024_D) = (1024_D) = 400_H \text{ (ubicaciones de memoria)}$$

$$N_{SR}^D = 4 \text{ bits (tamaño de cada ubicación de memoria)}$$

PASO ② Almacenamiento Requerido de los CI

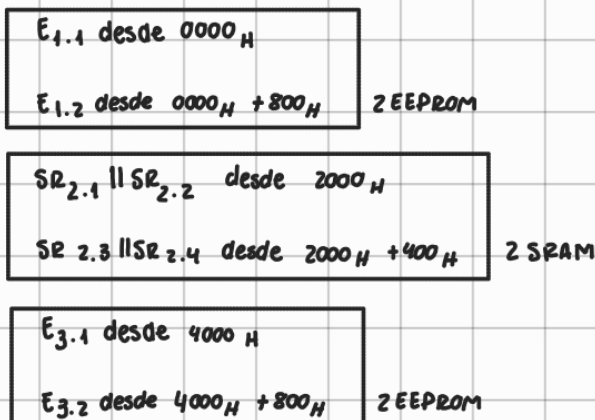
$$n_{CI(BANK1)} = \frac{C_E(REQ)}{C_E(DIS)} \cdot \frac{4K \times 8}{2K \times 8} = 2 \times 1E = 2 \text{ EEPROMs desde } 0000_H$$

$$n_{CI(BANK2)} = \frac{2K \times 8}{1K \times 4} = 2 \times 2SR = 4SRAM \text{ desde } 2000_H$$

$$n_{CI(BANK3)} = \frac{4K \times 8}{2K \times 8} = 2 \times 1E = 2 \text{ EEPROMs desde } 4000_H$$

PASO ③ Elaboración de Mapa de Memoria

como necesitamos 16 bits entonces



Cantidad de bits del bus de direcciones para direccionar cada memoria

$$n_E^A = \frac{\ln(N_E^A)}{\ln(2)} = 11 \rightarrow \langle A_{10} \dots A_0 \rangle$$

$$n_{SR}^A = \frac{\ln(N_{SR}^A)}{\ln(2)} = 10 \rightarrow \langle A_9 \dots A_0 \rangle$$

PASO ④ Completar mapa de memoria.

4.1 Completar el bus de direcciones con n^{a} del paso 3 en función de los rangos que ocupen cada memoria.

4.2 Ubicar los primeros 4 bits y verificar si hay columnas que no presenten cambios.

4.3 Determinar el tipo de mapeo : Absoluto : A_{15} y A_{12} en cero , las memorias no se reflejan en posiciones prohibidas

Relativo: A_{15} y A_{12} toman todos los valores posibles, las memorias pueden reflejarse en otras posiciones de memoria dentro de la zona prohibida

[illegible][illegible][illegible][illegible]

PASO ⑤ Decodificadores Requeridos

Siempre usamos Decos 3 a 8 (con salidas negadas) **74LS138**

$$n_{DECO} = \frac{n_{sys}^A - \min(n_E^A, n_{SR}^A)}{n'_{DECO}}$$

nro entradas del deco

$$n_{DECO} = \frac{16 - \min(11, 10)}{3} = 2 \text{ [Decos]}$$

PASO ⑥ Tabla de Decodificación

Ver los bits A_{14}, A_{13}, A_{11} y A_{10} para la tabla

A_{14}, A_{13}, A_{11} direccionan las EEPROM'S por lo tanto serán parte del

DECO #1, A_{10} es LSB

#1	A_{14}	A_{13}	A_{11}	$O_{X(1)}$
$E_{1.1}$	0	0	0	$O_{0(1)}$
$E_{1.2}$	0	0	1	$O_{1(1)}$
R	0	1	0	$O_{2(1)}$
	0	1	1	$O_{3(1)}$
$E_{3.1}$	1	0	0	$O_{4(1)}$
$E_{3.2}$	1	0	1	$O_{5(1)}$
	1	1	0	$O_{6(1)}$
	1	1	1	$O_{7(1)}$

El bit A_{10} y la salida 2 del **DECO #1**

direccionan las SRAM'S, en el **DECO #2** A_{10} es LSB

#2	GND	$O_{2(1)}$	A_{10}	$O_{X(2)}$
$R_{2.1} \parallel R_{2.2}$	0	0	0	$O_{0(2)}$
$R_{2.3} \parallel R_{2.4}$	0	0	1	$O_{1(2)}$
	0	1	0	$O_{2(2)}$
	0	1	1	$O_{3(2)}$
	1	0	0	$O_{4(2)}$
	1	0	1	$O_{5(2)}$
	1	1	0	$O_{6(2)}$
	1	1	1	$O_{7(2)}$

Datasheet de 74LS138

Function Tables

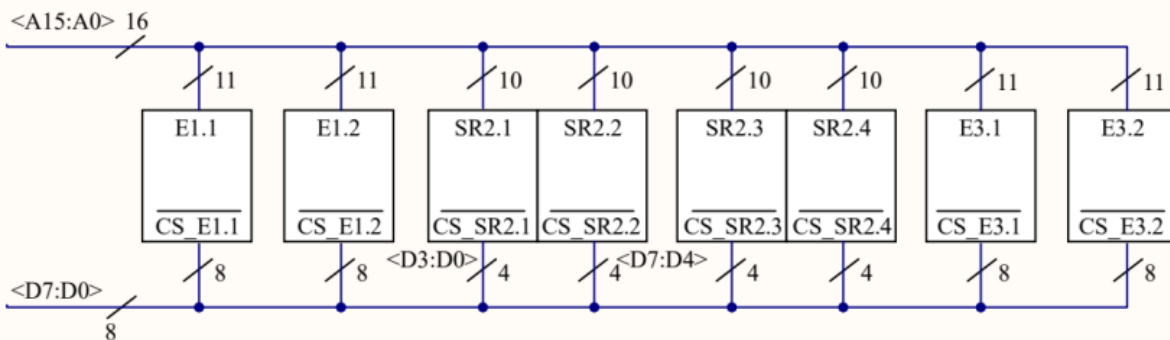
DM74LS138

Inputs		Outputs							
Enable		Select							
G1	G2 (Note 1)	C	B	A	Y0	Y1	Y2	Y3	Y4
X	H	X	X	X	H	H	H	H	H
L	X	X	X	X	H	H	H	H	H
H	L	L	L	L	H	H	H	H	H
H	L	L	L	H	L	H	H	H	H
H	L	L	H	L	H	L	H	H	H
H	L	L	H	H	H	L	H	H	H
H	L	H	L	L	H	H	L	H	H
H	L	H	L	H	H	H	L	H	H
H	L	H	H	L	H	H	H	L	H
H	L	H	H	H	H	H	H	L	H

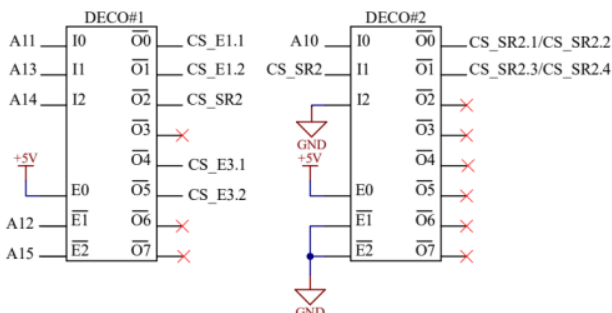
H = HIGH Level
L = LOW Level
X = Don't Care

Note 1: G2 = G2A + G2B

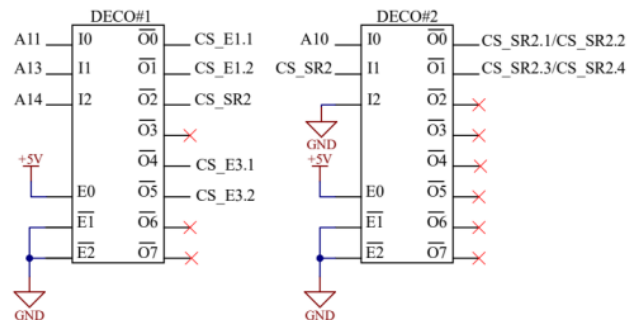
PASO ⑦ Graficar Hardware



MAPEO ABSOLUTO



MAPEO RELATIVO



Ejercicio 2.2 Guía Mapeo Directo de Memorias

Implementar un Banco de Memoria que responda al siguiente mapa:
 a partir de 0000h $\Rightarrow$ 8 KB de EPROM
 a partir de 3000h $\Rightarrow$ 4 KB de RAM
 a partir de 4000h $\Rightarrow$ 16 KB de EPROM
 a partir de A000h $\Rightarrow$ 4 KB de EPROM
 Se dispone de memorias RAM de 1K x 4 bits y EPROM de 4K x 8 bits.

PASO ① Almacenamiento disponible en los CI

$$EE : 4K \times 8 \rightarrow 4 \cdot (1024_D) = 4096_D \approx 1000_H \rightarrow N_H^A = 1000_H \text{ (ubicaciones de memoria)}$$

$$\rightarrow 8 \text{ bits} = N_E^D \text{ (tamaño de ubicación de memoria)}$$

$$RAM : 1K \times 4 \rightarrow 1 \cdot (1024_D) = 1024_D \approx 400_H \rightarrow N_H^A = 400_H \text{ (ubicaciones de memoria)}$$

$$\rightarrow 4 \text{ bits} = N_E^D \text{ (tamaño de ubicación de memoria)}$$

PASO ② Almacenamiento Requerido de los CI

$$n_{CI}(BANK1) = \frac{C_E(REQ)}{C_E(DISP)} = \frac{8K \times 8}{4K \times 8} = 2 \times 1E = 2 \text{ EPROMs desde } 0000_H$$

$$n_{CI}(BANK2) = \frac{4K \times 8}{1K \times 4} = 4 \times 2R = 8 \text{ RAMs desde } 3000_H$$

$$n_{CI}(BANK3) = \frac{16K \times 8}{4K \times 8} = 4 \times 1E = 4 \text{ EPROMs desde } 4000_H$$

$$n_{CI}(BANK4) = \frac{4K \times 8}{4K \times 8} = 1 \times 1E = 1 \text{ EPROM desde } A000_H$$

PASO ③ Elaboración de Mapa de Memoria Rango : Dirección inicial + tamaño - 1

como necesito 16 bits:

BANCO 1

BANCO 3

$$\begin{aligned} E_{1.1} &\rightarrow 0000_H \text{ a } 0FFF_H \\ E_{1.2} &\rightarrow 1000_H \text{ a } 1FFF_H \end{aligned}$$

$$\begin{aligned} E_{2.1} &\rightarrow 4000_H \rightarrow 4FFF_H \\ E_{2.2} &\rightarrow 5000_H \rightarrow 5FFF_H \\ E_{2.3} &\rightarrow 6000_H \rightarrow 6FFF_H \\ E_{2.4} &\rightarrow 7000_H \rightarrow 7FFF_H \end{aligned}$$

Cantidad de bits del bus de direcciones para direccionar cada memoria

$$n_E^A = \frac{\ln(N_E^A)}{\ln(2)} = 12 \rightarrow \langle A_{11} \dots A_0 \rangle$$

$$n_R^A = \frac{\ln(N_R^A)}{\ln(2)} = 10 \rightarrow \langle A_9 \dots A_0 \rangle$$

$$\begin{aligned} R_{1.1} \parallel R_{1.2} & 3000_H \rightarrow 33FF_H \\ R_{1.3} \parallel R_{1.4} & 3400_H \rightarrow 37FF_H \\ R_{1.5} \parallel R_{1.6} & 3800_H \rightarrow 3BFF_H \\ R_{1.7} \parallel R_{1.8} & 3C00_H \rightarrow 3FFF_H \end{aligned}$$

BANCO 2

BANCO 4

$$E_3 \quad A000_H \rightarrow AFFF_H$$

PASO ④ Completar mapa de memoria

CI	Rango [H]	A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
E1.1	0000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	0FFF	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
E1.2	1000	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
	1FFF	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1
R1.1	3000	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0
R1.2	33FF	0	0	1	1	0	0	1	1	1	1	1	1	1	1	1	1
R1.3	3400	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0
R1.4	37FF	0	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1
R1.5	3800	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0
R1.6	38FF	0	0	1	1	1	0	1	1	1	1	1	1	1	1	1	1
R1.7	3600	0	0	1	1	0	1	1	0	0	0	0	0	0	0	0	0
R1.8	3FFF	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
E2.1	4000	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	4FFF	0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1
E2.2	5000	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0
	5FFF	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1
E2.3	6000	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
	6FFF	0	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1
E2.4	7000	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0
	7FFF	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
E3	A000	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
	AFFF	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1

4.1

4.2

4.3. Como el enunciado no aclara nada seguimos con mapa absoluto y conectaremos todo en el deco.

PASO ⑤ Decodificadores Requeridos (74LS138)

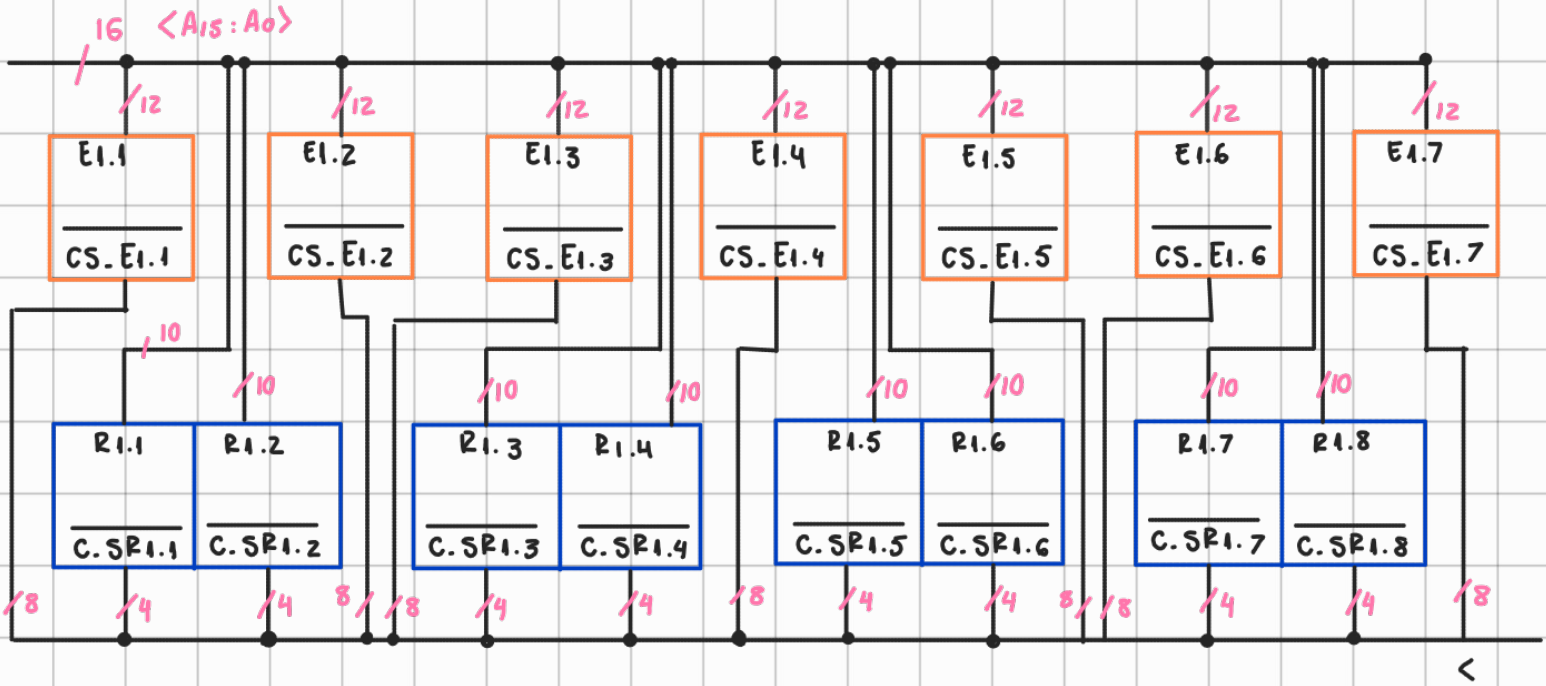
$$n_{\text{deco}} = \frac{16 - \min(12, 10)}{3} = \frac{16 - 10}{3} = 2 \text{ [decos]}$$

PASO ⑥ Tabla de Decodificación

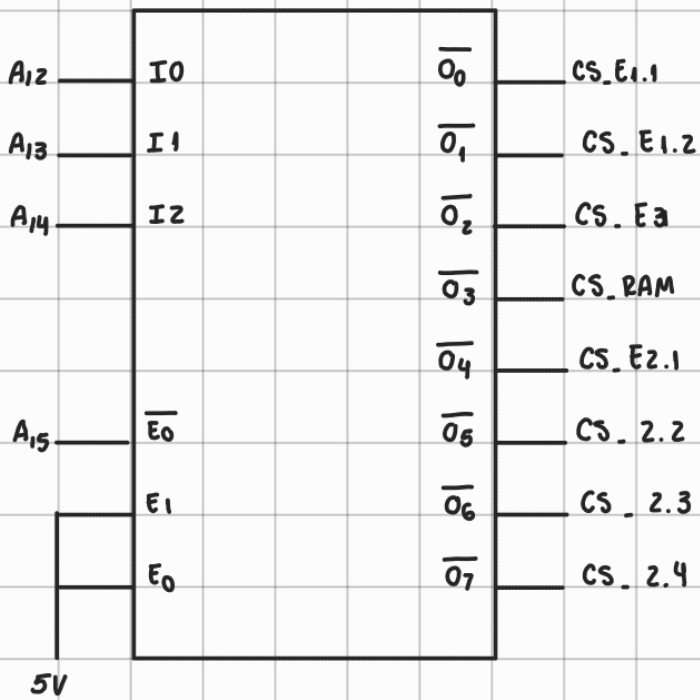
DECO 1	A14	A13	A12	Ox (1)
E1.1	0	0	0	08(1)
E1.2	0	0	1	01(1)
E3	0	1	0	02(1)
DECO 2	0	1	1	03(1)
E2.1	1	0	0	04(1)
E2.2	1	0	1	05(1)
E2.3	1	1	0	06(1)
E2.4	1	1	1	07(1)

DECO 2	GND	A11	A10	Ox (2)
R1.1 // R1.2	0	0	0	08(2)
R1.3 // R1.4	0	0	1	01(2)
R1.5 // R1.6	0	1	0	02(2)
R1.7 // R1.8	0	1	1	03(2)
				04(2)
				05(2)
				06(2)
				07(2)

PASO ⑦ Dibujar Hardware



DECO #4



DECO #	2
--------	---

